

基础垃圾文本检测

- 垃圾文本数据集
 - enron-spam
 - chinese text classification
 - twitter social honeypot
- 数据预处理
 - 去掉符号空格、数字、HTML/XML标记
 - 正则表达式 P21
 - 检测空白文本、错误编码、意外字符
 - 标记化（将文本分割为单词 jieba库）
 - 去掉停用词（无意义的词，如“的”）
 - 哈工大停用词表
 - 百度停用词表
 - 中文停用词表
 - SCU停用词表
- 特征提取
 - 词袋
 - TF-IDF
 - N-gram
 - word2vec
- 文本分类
 - 朴素贝叶斯
 - 支持向量机
 - 逻辑回归（拟合分类的曲线）
 - 深度学习
- 模型评估 混淆矩阵
- 调优和优化 P66
 - 更复杂的特征提取
 - 样本平衡（过采样、欠采样）
 - 分类模型调参

文本检测优化

字形、字音、字符相似性

(4) 基于字形编码的字形相似性



□ 函数的主要步骤如下：

(1) 设置一个特征权重weights，用于对不同特征的相似性给予不同的重要性（注意此处权重分配与文中不同）。

(2) 遍历字形编码中的每个特征，对于前featureSize-1个特征：如果两个字形编码的对应特征相同，就将其相似性乘以1；如果特征不同，就将其相似性乘以0。

(3) 根据特征的权重和相似性计算字形编码的总相似度shapeSimilarity。

```
1. # 计算字形相似性的函数
2. def computeShapeCodeSimilarity(shapeCode1, shapeCode2):
3.     # 特征大小(字形编码的长度)
4.     featureSize=len(shapeCode1)
5.     # 特征权重
6.     weights=[0.15,0.15,0.15,0.15,0.15,0.15,0.1]
7.     multiplier=[]
8.     # 计算字形编码的相似性
9.     for i in range(featureSize):
10.         if shapeCode1[i]==shapeCode2[i]:
11.             multiplier.append(1)
12.         else:
13.             multiplier.append(0)
14.     shapeSimilarity=0
15.     # 计算字形编码的相似性
16.     for i in range(featureSize):
17.         shapeSimilarity += weights[i]*multiplier[i]
18.     return shapeSimilarity
```

83

(2) 基于字音编码的字音相似性



□ 字音编码代码：

在上式中，最重要的是声母和韵母，补码以及声调都是次要的。以下是一个计算两个汉字字音相似性的示例代码：

```
def computeSoundCodeSimilarity(soundCode1, soundCode2):
    featureSize=len(soundCode1)
    weights=[0.4, 0.4, 0.1, 0.1]
    multiplier=[]
    for i in range(featureSize):
        if soundCode1[i]==soundCode2[i]:
            multiplier.append(1)
        else:
            multiplier.append(0)
    soundSimilarity=0
    for i in range(featureSize):
        soundSimilarity += weights[i]*multiplier[i]
    return soundSimilarity
```

1) 接收两个字音编码soundCode1和soundCode2作为输入函数。

2) 确定字音编码的特征大小(即编码长度)，并初始化特征权重列表weights。

3) 对于每个特征，比较soundCode1和soundCode2中对应位置的元素，如果相同，则将相似度乘数置为1，否则置为0，存储在multiplier列表中。

4) 将每个特征的加权相似度相加，得到字音编码的总相似度soundSimilarity。

2.3 字符相似性网络



□ 字符相似性代码：

```
1. # 计算字符相似性的函数
2. def computeSSCSimilarity(ssc1, ssc2):
3.     # 组合字音和字形的相似性，根据权重计算
4.     shapeSimi=computeShapeCodeSimilarity(ssc1[4:], ssc2[4:])
5.     soundSimi=computeSoundCodeSimilarity(ssc1[:4], ssc2[:4])
6.     return max(soundSimi, shapeSimi)
```

在处理字形相似性和字音相似性时，选择max聚合，而不是平均聚合或加权聚合。这是因为在实际应用中，**大部分变体**都只简单地考虑字形或字音的**其中一个角度**，**并不会采取两者的结合**。

课堂QA：

1. 除了上述计算字形相似性，请思考其他的字形相似性计算方法
2. 除了上述计算字音相似性，请思考其他的字音相似性计算方法
3. 有没有更好的字符相似性计算机方法？

实践示范 P91